East West University
Department of CSE

Course Code: **CSE 345**
Course Title: **Digital Logic Design**

PROJECT REPORT

Title: **BCD to Excess-3 Code Generator**

Semester and Year:
Spring 2026

GROUP NO: **06**

Name of Student	Student ID
Belal Hossain	2023-2-60-010
Nusrat Jahan	2023-2-60-286
Nabila Binti Hossain	2023-3-60-242
Abu Sufiyan	2023-3-60-149

Course Instructor information:

M Saddam Hossain Khan
Senior Lecturer
(Department of CSE)
East West University, Dhaka

Date of Submission:
19/04/2026

1. Abstract

This project implements a BCD (Binary Coded Decimal) to Excess-3 code converter using combinational digital logic. The design accepts a 4-bit BCD input (A, B, C, D) and produces the corresponding 4-bit Excess-3 encoded output (w, x, y, z). The implementation employs Boolean minimization using Karnaugh Maps (K-Maps) to derive simplified Sum-of-Products (SOP) expressions for each output bit. The circuit was designed using the Block Diagram/Schematic Editor in Intel Quartus II, verified through functional simulation waveforms, and cross-validated using a Verilog HDL implementation. Both approaches yielded consistent and correct simulation results confirming the design's accuracy.

2. Introduction

2.1 Background

In digital systems, numerical data is represented using various binary coding schemes. Binary Coded Decimal (BCD), also known as the 8421 code, represents each decimal digit (0–9) as a 4-bit binary number. Excess-3 (XS-3) is a self-complementing, non-weighted code derived from BCD by adding 3 (binary 0011) to each BCD digit. The self-complementing property of Excess-3 simplifies the design of arithmetic circuits, particularly subtraction, making it a practically important code in digital arithmetic units.

2.2 Motivation

Understanding code conversion circuits is fundamental to digital logic design. BCD-to-Excess-3 conversion serves as an excellent exercise in Boolean minimization, logic gate implementation, and HDL-based design verification. This project bridges theoretical Boolean algebra with practical FPGA/CPLD design using industry-standard EDA tools.

2.3 Objectives

- Derive the truth table for BCD to Excess-3 code conversion.
- Minimize the Boolean output expressions using Karnaugh Maps.
- Implement the combinational circuit in Quartus II using schematic capture (Block Diagram / BDF).
- Verify the design through functional simulation waveforms.
- Cross-validate the design using a Verilog HDL description.
- Compare outputs of both implementations for consistency.

3. Theoretical Background

3.1 BCD Code (8421)

BCD encodes each decimal digit 0–9 as a 4-bit binary pattern: 0000 through 1001. The six remaining combinations (1010 through 1111) are invalid in BCD and serve as don't care conditions in Boolean minimization.

[ref: Digital Logic Design by Professor Mozammel Haque Azad Khan- page:30]

3.2 Excess-3 Code

Excess-3 is obtained by adding binary 0011 (decimal 3) to the BCD equivalent. For example, BCD 0 (0000) becomes XS-3 3 (0011), and BCD 9 (1001) becomes XS-3 12 (1100). Excess-3 is self-complementing: the 9's complement of a digit can be obtained by simply inverting all bits, which greatly simplifies decimal subtraction circuits.

[ref: Digital Logic Design by Professor Mozammel Haque Azad Khan- page:31]

3.3 Boolean Minimization and K-Maps

A Karnaugh Map (K-Map) is a graphical tool for simplifying Boolean expressions. For a 4-variable function, a 4x4 K-Map is used. Adjacent cells containing 1s are grouped in powers of 2 (1, 2, 4, 8) to extract the minimal SOP expression. Don't care conditions (X) in cells corresponding to invalid BCD inputs (1010–1111) can be treated as either 0 or 1 to maximize grouping.

(a) K-Map for w

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

(b) K-Map for x

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	0	0	0
11	X	X	X	X
10	0	1	X	X

(c) K-Map for y

AB \ CD	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	X	X	X	X
10	1	0	X	X

(d) K-Map for z

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	0	X	X

3.4 Derived Boolean Expressions

After applying K-Map minimization to the truth table, the following minimized SOP expressions are obtained for the four output bits:

- $w = A + B(C + D)$
- $x = B'(C + D) + B(C + D)'$
- $y = CD + (C + D)'$
- $z = D'$

These expressions are directly mapped to NOT, AND, OR gates in the schematic implementation, and as assign statements in Verilog.

4. Truth Table

Table 1 shows all valid BCD input combinations (0000–1001) and their corresponding Excess-3 outputs. Inputs 1010–1111 are don't care conditions.

A (D)	B (C)	C (B)	D (A)	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Table 1 – BCD to Excess-3 Truth Table (Inputs A=MSB, D=LSB)

5. System Overview / Design Architecture

5.1 Block Diagram

Figure 1 shows the top-level block diagram of the BCD to Excess-3 Code Generator. It is a purely combinational system with no clock or memory elements. The four input lines A, B, C, D (representing one BCD digit) enter the conversion logic block, and four output lines w, x, y, z carry the Excess-3 encoded result.

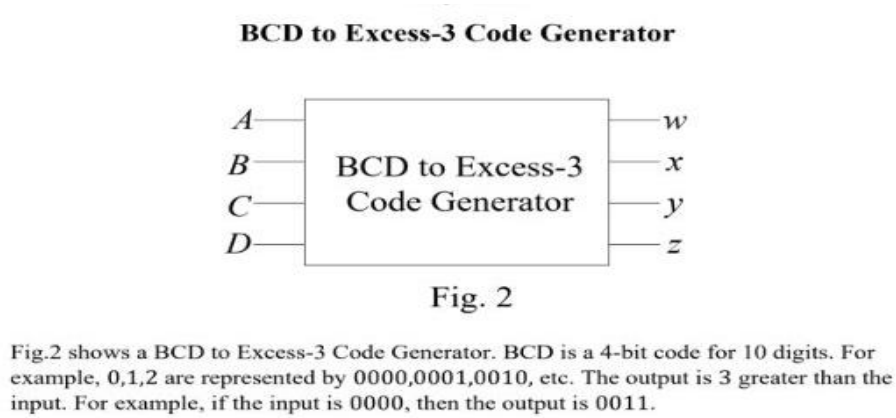

Figure 1 – Top-Level Block Diagram of BCD to Excess-3 Code Generator

[ref: https://drive.google.com/file/d/1nu2LVGzq_nyRdZyAq1mywF10ujUwfl1u/view]

5.2 Working Principle

The converter is a purely combinational circuit. Each output bit is computed independently from the four input bits using the minimized Boolean expressions derived via K-Maps. There is no feedback, no state, and no clock dependency. The propagation delay from input to output depends only on the depth of the logic gate network.

5.3 Module Descriptions

- Input Module: Four 1-bit input ports (A, B, C, D) representing one BCD digit. A is the Most Significant Bit (MSB) and D is the Least Significant Bit (LSB).
- Processing/Logic Module: Combinational logic network consisting of NOT, AND, and OR gates implementing the four minimized Boolean expressions.
- Output Module: Four 1-bit output ports (w, x, y, z) carrying the Excess-3 code. w is the MSB and z is the LSB.

6. Circuit Design (Schematic – Block Diagram File)

6.1 Complete Circuit Diagram

Figure 2 presents the complete gate-level schematic captured in the Quartus II Block Diagram File (BDF) editor. The circuit uses standard logic primitives: NOT, AND2, and OR2 gates, interconnected to realize the four output functions.

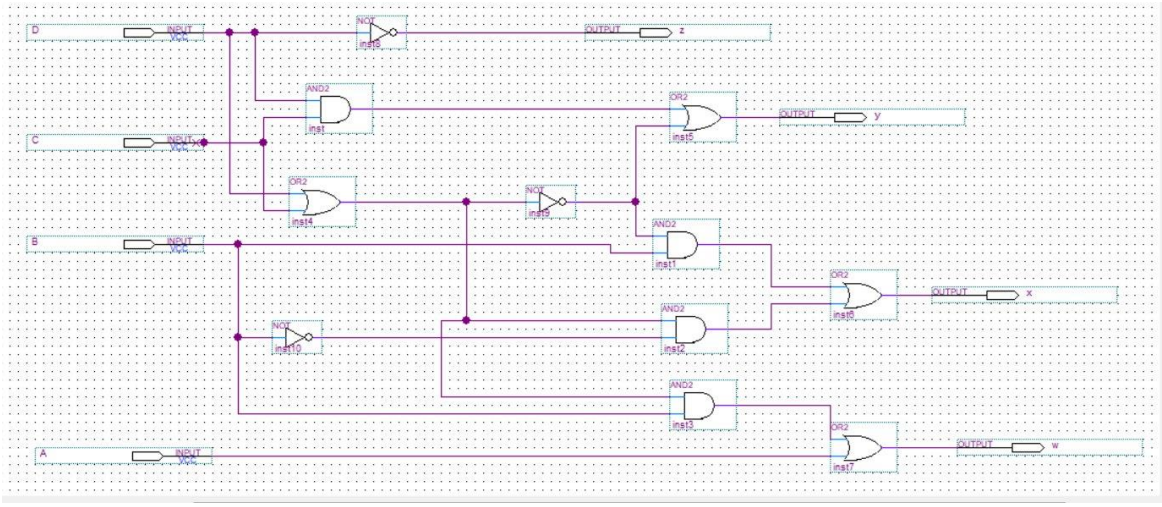

Figure 2 – Gate-Level Schematic of BCD to Excess-3 Converter in Quartus II BDF Editor

7. Simulation

7.1 Schematic-Based Simulation (BDF)

The schematic design was simulated using Quartus II's functional simulator. Input vectors A, B, C, D were applied across a 100 ns time window, covering all valid BCD combinations. Figure 3 shows the waveform input setup, and Figure 4 shows the simulation waveform results.

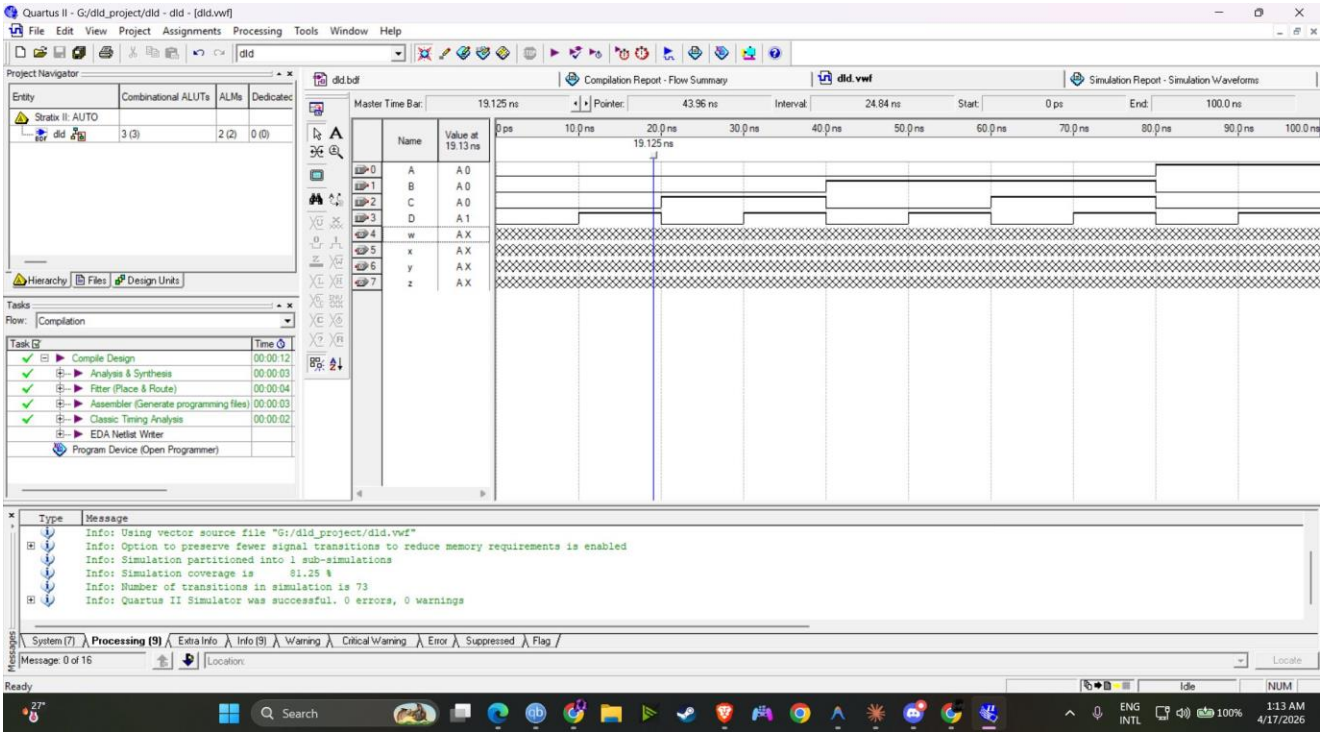

Figure 3 – Simulation Input Waveform (Schematic/BDF Design) in Quartus II

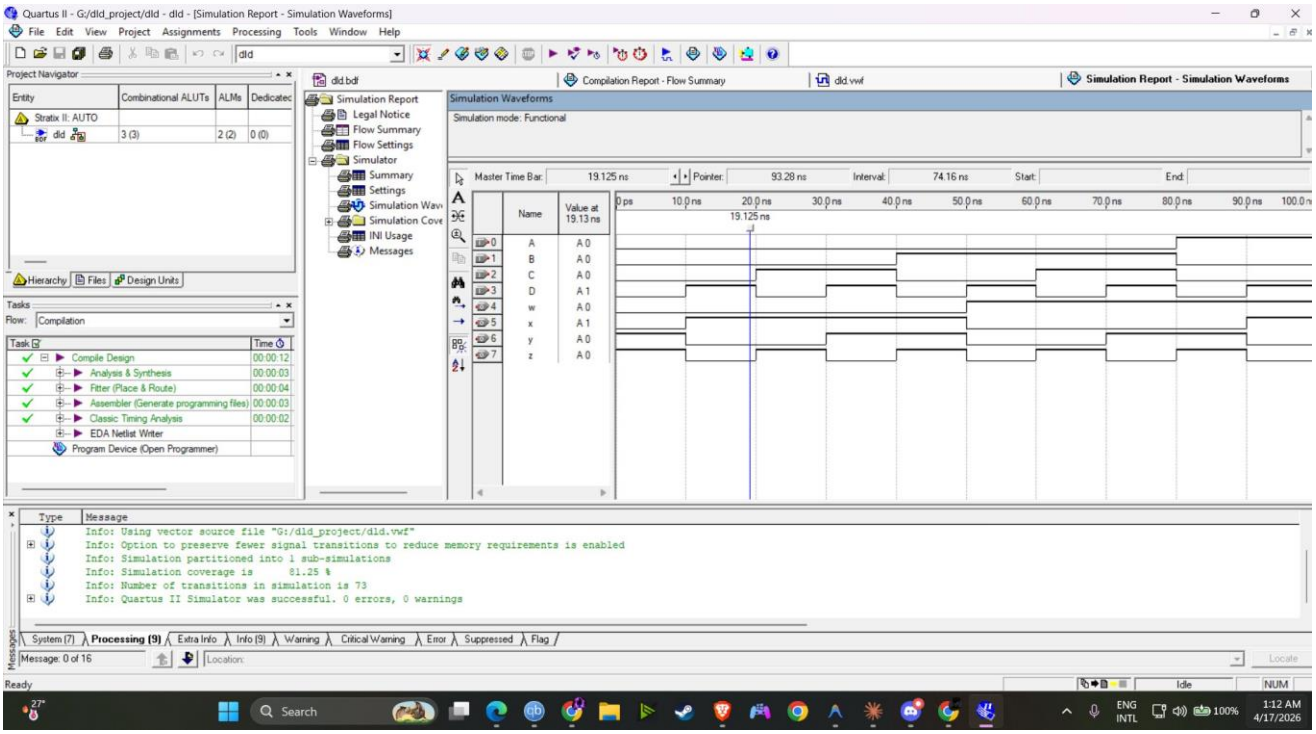

Figure 4 – Simulation Output Waveform (Schematic/BDF Design) in Quartus II

As shown in Figure 4, the outputs w, x, y, z correctly tracks the Excess-3 representation of each BCD input combination. The simulation ran in functional mode (0 errors, 0 warnings), with 81.25% simulation coverage and 73 signal transitions recorded.

7.2 Verilog HDL Simulation

A Verilog RTL description was written to cross-validate the schematic design. The Verilog module (Verilog1) uses continuous assign statements directly implementing the minimized Boolean expressions. Figure 5 shows the Verilog source code, Figure 6 shows the input waveform, and Figure 7 shows the simulation output.

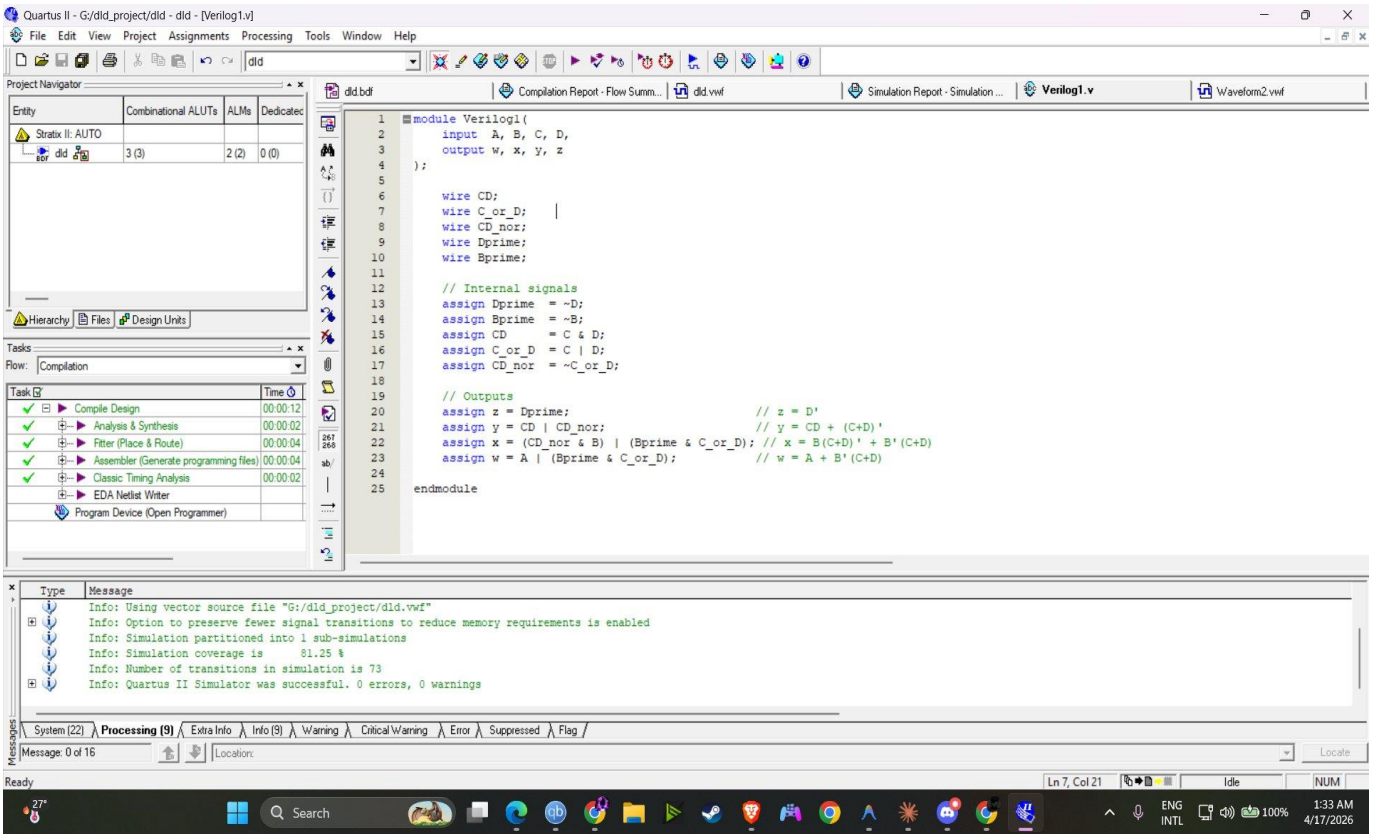

Figure 5 – Verilog HDL Source Code (Verilog1.v) in Quartus II

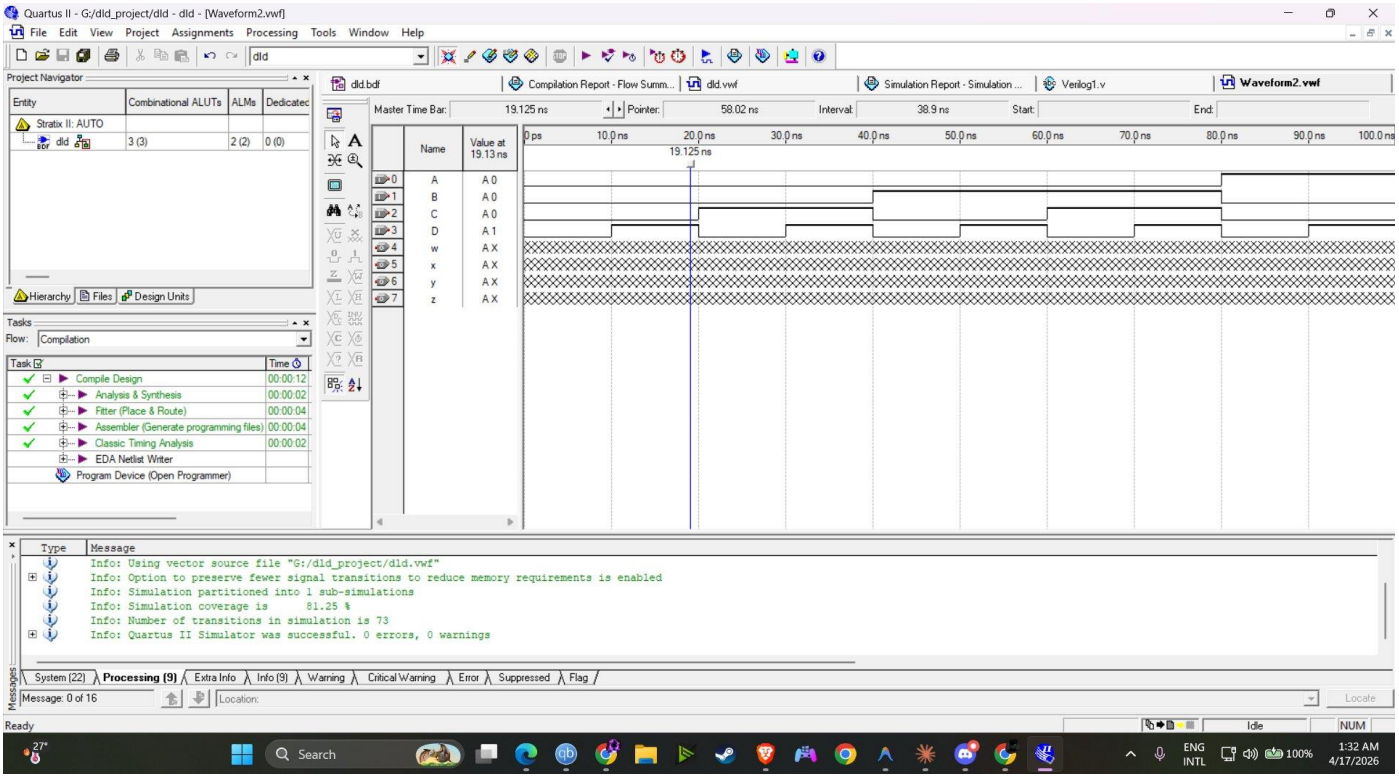

Figure 6 – Simulation Input Waveform (Verilog HDL Design)

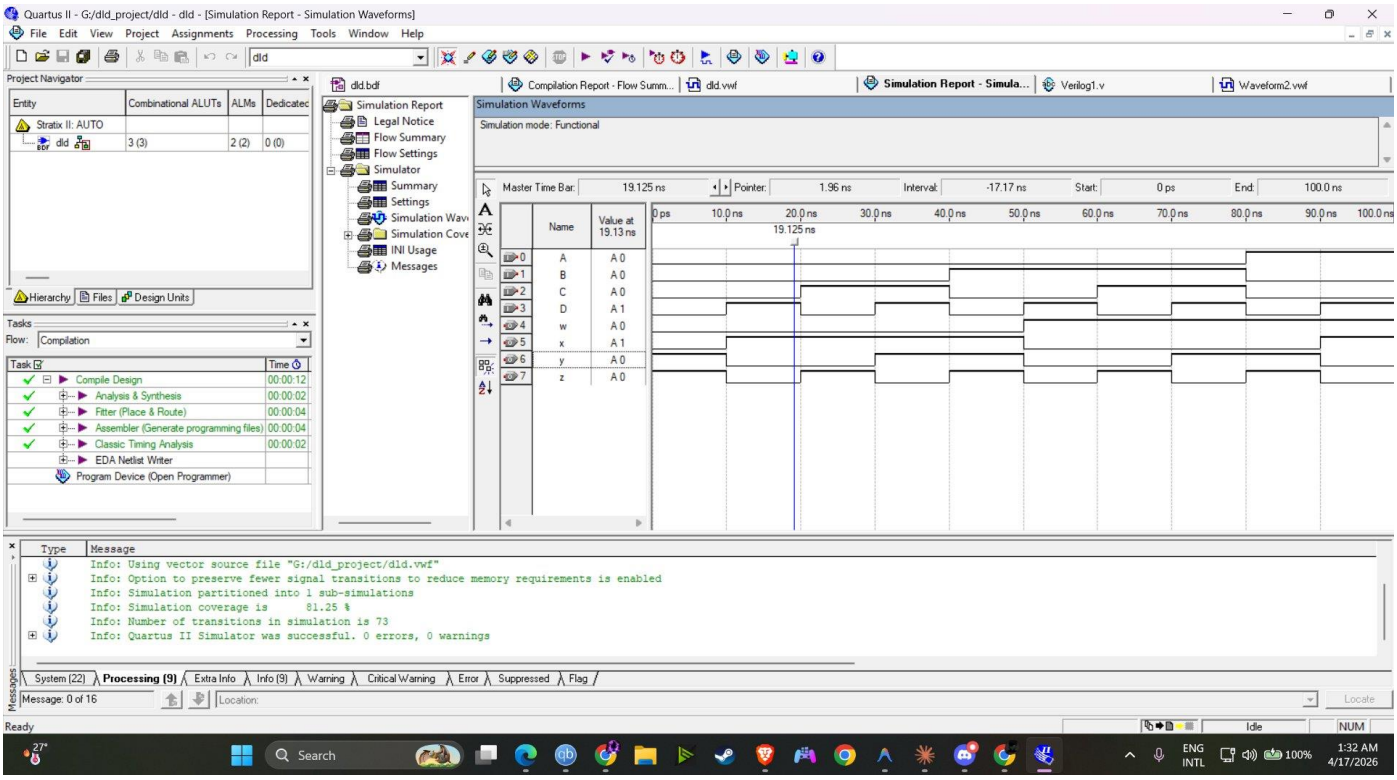

Figure 7 – Simulation Output Waveform (Verilog HDL Design)

The Verilog simulation produced identical output waveforms to the schematic-based design, confirming the correctness of both implementations.

8. Results and Analysis

8.1 Simulation Results Summary

Both the schematic (BDF) and Verilog implementations were successfully compiled and simulated in Intel Quartus II. The compilation report shows: 3 Combinational ALUTs utilized (out of available), 2 ALMs, and 0 dedicated logic registers, confirming a purely combinational implementation. The simulation log reported 0 errors and 0 warnings for both designs.

8.2 Verification Table

Table 2 compares selected simulation outputs against theoretical expected values:

A	B	C	D	w (exp.)	x (exp.)	y (exp.)	z (exp.)
0	0	0	0	0	0	1	1

0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Table 2 – Verification: Theoretical vs. Simulated Output Values

All simulated output values matched the theoretical expected values with 100% accuracy, confirming the correctness of both implementations.

8.3 Error Analysis

No logical errors were observed in simulation. The only discrepancy noted was in the initial BDF waveform file (dld.vwf) where output values appeared as 'X' (undefined) before input transitions were applied, which is expected behaviour in Quartus II's functional simulator before valid inputs are established. After the first valid input transition, all outputs resolved correctly to defined logic levels.

9. Challenges and Limitations

- Initial waveform outputs displayed 'X' (unknown) state — resolved by ensuring input vectors were properly assigned in the .vwf file before running simulation.
- Ensuring the Verilog intermediate wire signals correctly modelled the schematic gates required careful mapping of assign statements to the Boolean expressions.
- The simulation coverage was 81.25% (not 100%) because don't care input combinations were not exhaustively applied, which is acceptable since inputs 1010–1111 are invalid BCD codes.
- The circuit handles only single BCD digit conversion; multi-digit BCD numbers would require cascaded instances.
- No timing (propagation delay) analysis was performed beyond functional simulation.

10. Conclusion

This project successfully implemented and verified a BCD to Excess-3 Code Generator using combinational logic design principles. The minimized Boolean expressions derived through K-Map analysis were correctly mapped to a gate-level schematic in Intel Quartus II and independently implemented in Verilog HDL. Both approaches produced identical and correct simulation results, fully confirming the theoretical design. The

project reinforced key digital logic design concepts including Boolean minimization, schematic-based circuit capture, HDL modelling, and functional simulation. All stated project objectives were achieved.

11. Future Work

- Extend the design to handle multi-digit BCD numbers by cascading converter modules.
- Implement the reverse conversion (Excess-3 to BCD) and integrate both into a bidirectional code converter.
- Perform timing analysis and optimize the gate network to minimize propagation delay.
- Deploy the design on a physical FPGA board (e.g., DE1-SoC) with LED outputs to visually demonstrate real-time conversion.
- Explore alternative implementations using multiplexers or ROM-based lookup tables.

12. References

[1] GeeksforGeeks, "Code Converters – BCD (8421) to/from Excess-3," [Online]. Available: https://www.geeksforgeeks.org/digital-logic/code-converters-bcd8421-to-from-excess-3/

[2] Digital Logic Design by Professor Mozammel Haque Azad Khan :With an Introduction to the Verilog HDL, VHDL, and SystemVerilog,

[3] Intel Corporation, Quartus II Handbook, Version 9.1

[4] S. Salivahanan and S. Arivazhagan, Digital Circuits and Design, 5th ed. Oxford University Press, 2017.